Using the Arduino UNO R4 WiFi LED Matrix

Get off the ground with the Arduino UNO R4 WiFi built-in LED matrix. Learn the different techniques for controlling it, create animations, graphics or even games.

Author Jacob Hylén & Tom Igoe · Last revision 10/07/2025

The **Arduino UNO R4 WiFi** comes with a built in 12x8 LED Matrix, that is available to be programmed to display graphics, animations, act as an interface, or even play games on.

Goals

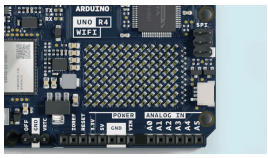
The matrix and its API are developed to be programmed in a few different ways, each suited for different applications. This guide will walk you through the basic concepts for programming the LED matrix, and get you started with creating your own animations, while highlighting two different ways of handling the LEDs to create animations and images.

ON THIS PAGE

- Goals
- Hardware & Software Needed
- Initializing Matrix
- How to Write a Frame
- Testing It Out
- Resources
- Scrolling Text Example
- Frame Gallery
- Animation Generation
- API
- Members
  - ArduinoLEDMatrix()
  - autoscroll()
  - begin()
  - next()
  - loadFrame()
  - renderFrame()
  - loadSequence()
  - play()
  - sequenceDone()

Help

decide what method fits your needs best!



The UNO R4 WiFi LED Matrix

Hardware & Software Needed

- **Arduino UNO R4 WiFi**
- **UNO R4 Board Package** (latest version)
- **Arduino IDE**

Initializing Matrix

To use the LED Matrix library, there are a few things that need to be added to your sketch to get off the ground.

First, include the library at the top of your sketch, like this:

```
#include "Arduino_LED_
```

Then, you'll need to create a LED Matrix object in your sketch, by adding the following line directly underneath the first one:

```
ArduinoLEDMatrix matri
```

And then lastly, start the LED Matrix by adding this line in `void setup()`:

```
matrix.begin();
```

The entire thing should look like this;

```
#include "Arduino_LED_

ArduinoLEDMatrix matri

void setup() {
  Serial.begin(115200)
  matrix.begin();
}
```

How to Write a Frame

The LED Matrix library for the UNO R4 WiFi works on the principle of creating a frame, and then loading it into a buffer which displays the frame.

A frame is what we call the "image" that is displayed at any given moment on the matrix. If an animation is a series of images, a frame is one of those

In order to control the 12x8 LED matrix on the UNO R4 WiFi, you need a space in memory that's at least 96 bits in size. The library provides two ways to do this.

The first is simply to make a two-dimensional array of bytes like so:

```
byte frame[8][12] =
  { 0, 0, 1, 1, 0, 0,
  { 0, 1, 0, 0, 1, 0,
  { 0, 1, 0, 0, 0, 1,
  { 0, 0, 1, 0, 0, 0,
  { 0, 0, 0, 1, 0, 0,
  { 0, 0, 0, 0, 1, 0,
  { 0, 0, 0, 0, 0, 1,
  { 0, 0, 0, 0, 0, 0,
};
```

This option is simple to understand, because you can see the image in the pattern of the array, and it is easy to edit in runtime. The ones in the array above form a heart, and that's the image you'd see on the screen.

To target an individual pixel you select its address and change the value, remember that you'll need to start counting at 0. So, the following line will target the third pixel from the top and the second from the left, then turn it on:

```
frame[2][1] = 1;

matrix.renderBitmap(fr
```

This method takes more memory than is needed, however. Even though each LED needs only a single bit to store its state, you're using eight bits (a byte). The more memory-efficient method to store a frame is to use an array of 32-bit integers.

In this section we'll walk through the process and the concept of how you may create one of these frames yourself. Though we have developed a tool that can do this for you, so you can **click here** if you want to skip this exercise.

Here's the same heart in that form:

```
unsigned long frame[]
  0x3184a444,
  0x42081100,
  0xa0040000
};
```

An unsigned long variable holds 32 bits, and 96/32 is 3, so an unsigned long array is an efficient way to hold all the bits you need for the LED matrix.

But how do those hexadecimal values relate to the positions of

binary values. Here's a code snippet that will do this:

```
for (int b = 0; b < 3
    Serial.println(fra
  }
```

This will print out all the bit values of the array. The output will look like this:

```
11000110000100101001000
10000100000100000010000
10100000000001000000000
```

This method doesn't show you all the bits, though. Each array element should have 32 bits. If you add zeros to show all 32 bits of each element, you get:

```
00110001100001001010010
01000010000010000000100
10100000000001000000000
```

Now divide it into groups of 12 bits and you've got the heart back:

```
001100011000
010010100100
010001000100
001000001000
000100010000
000010100000
000001000000
000000000000
```

Hint: You can see the heart easier if you highlight all "1"s on the page by pressing CTRL/command + F and search for "1".

If you've got several different frames, you can load and display them like this:

```
const uint32_t happy
    0x19819,
    0x80000001,
    0x81f8000
};

const uint32_t heart
    0x3184a444,
    0x44042081,
    0x100a0040
};

  matrix.loadFrame(ha
  delay(500);

  matrix.loadFrame(he
  delay(500);
```

Testing It Out

Let's apply these concepts, with two basic sketches that display different frames on your board. First, let's create 3x32-bit integer frames and load them one by one.

Here's a sketch that will first load a smiley face on your matrix, and then change it to a heart.

```
#include "Arduino_LL

ArduinoLEDMatrix mat

void setup() {
  Serial.begin(11526
  matrix.begin();
}

const uint32_t happy
    0x19819,
    0x80000001,
    0x81f8000
};
const uint32_t heart
    0x3184a444,
    0x44042081,
    0x100a0040
};

void loop(){
  matrix.loadFrame(h
  delay(500);

  matrix.loadFrame(h
  delay(500);
```

The sketch is pretty simple, and yet the outcome is very expressive and can help you easily indicate states of your projects.

Now let's change approach and create a bitmap that we change in runtime. This sketch includes several functions that each draw part of a face, and then winks the left eye by turning off certain pixels.

```
  { 0, 0, 0, 0, 0, 0
  { 0, 0, 0, 0, 0, 0
  { 0, 0, 0, 0, 0, 0
  { 0, 0, 0, 0, 0, 0
  { 0, 0, 0, 0, 0, 0
  { 0, 0, 0, 0, 0, 0
  { 0, 0, 0, 0, 0, 0
  { 0, 0, 0, 0, 0, 0
};

void leftEye(){
  //Left eye
  frame[1][3] = 1;
  frame[1][4] = 1;
  frame[2][3] = 1;
  frame[2][4] = 1;
}

void wink(){
  //Wink with the le
  frame[1][3] = 0;
  frame[1][4] = 0;
  frame[2][3] = 1;
  frame[2][4] = 1;
}

void rightEye(){
```

Resources

The rest of this article is a collection of resources, such as functioning examples, a gallery of frames, and tools that can help you get started with the

Scrolling Text Example

The LED Matrix now supports printing characters via the ArduinoGraphics library. With it, you are able to:

- Set a start location for the text via `matrix.beginText(x,y, 0xFFFFFF)`. The "0xFFFFFF" represents the default color (red). As the **ArduinoGraphics** library supports other hardware with multiple colors, we need to specify it.
- Print the text via `matrix.printText("This message is printed")`
- End the print and (optionally) specify scroll direction with `matrix.endText(direction)`
- `SCROLL_LEFT`, `SCROLL_RIGHT` are supported. Leave blank if no scroll is desired.

The example below simply prints out **"Hello World!"** on the matrix.

> For this example to work, you need to install the ArduinoGraphics library to enable character printing on the LED matrix using the library manager.

ARDUINODOCS

```
// To use ArduinoGra
#include "ArduinoGra
#include "Arduino_LL

ArduinoLEDMatrix mat

void setup() {
  Serial.begin(11520
  matrix.begin();

  matrix.beginDraw()
  matrix.stroke(0xFF
  // add some static
  // will only show
  const char text[]
  matrix.textFont(Fo
  matrix.beginText(0
  matrix.println(tex
  matrix.endText();

  matrix.endDraw();

  delay(2000);
}

void loop() {
```

Frame Gallery

We've designed a gallery of frames and animations that are included in the library! You may load and display them on your UNO R4 WiFi with the following code snippet:

```
#include "Arduino_LE

ArduinoLEDMatrix matr

void setup() {
  Serial.begin(115200
  matrix.begin();

}

void loop() {
  // Load and display
  matrix.loadFrame(LE
}
```

By changing the parameter in `matrix.loadFrame()` in the loop, you can choose among the available frames we designed.

The available frames are:

- `LEDMATRIX_BLUETOOTH`
- `LEDMATRIX_BOOTLOADER_ON`
- `LEDMATRIX_CHIP`
- `LEDMATRIX_CLOUD_WIFI`
- `LEDMATRIX_DANGER`
- `LEDMATRIX_EMOJI_BASIC`
- `LEDMATRIX_EMOJI_HAPPY`
- `LEDMATRIX_EMOJI_SAD`
- `LEDMATRIX_HEART_BIG`
- `LEDMATRIX_HEART_SMALL`
- `LEDMATRIX_LIKE`
- `LEDMATRIX_MUSIC_NOTE`
- `LEDMATRIX_RESISTOR`
- `LEDMATRIX_UNO`

Alternatively, play one of the animations on the LED matrix like this:

```
#include "Arduino_LE

// Create an instance
ArduinoLEDMatrix matr

void setup() {
  Serial.begin(115200
  // you can also loa
  matrix.loadSequence
  matrix.begin();
  matrix.play(true);
}

void loop() {
}
```

In this case, you change the parameter of `matrix.loadSequence()` in the setup to one of the available ones to display.

The available animations are:

- `LEDMATRIX_ANIMATION_STARTUP`
- `LEDMATRIX_ANIMATION_TETRIS_INTRO`
- `LEDMATRIX_ANIMATION_ATMEGA`
- `LEDMATRIX_ANIMATION_LED_BLINK_HORIZONTAL`
- `LEDMATRIX_ANIMATION_LED_BLINK_VERTICAL`
- `LEDMATRIX_ANIMATION_ARROWS_COMPASS`
- `LEDMATRIX_ANIMATION_AUDIO_WAVEFORM`
- `LEDMATRIX_ANIMATION_BATTERY`
- `LEDMATRIX_ANIMATION_BOUNCING_BALL`
- `LEDMATRIX_ANIMATION_BUG`
- `LEDMATRIX_ANIMATION_C`

`LEDMATRIX_ANIMATION_CLOUD`

`LEDMATRIX_ANIMATION_DOWNLOAD`

`LEDMATRIX_ANIMATION_DVD`

`LEDMATRIX_ANIMATION_HEARTBEAT_LINE`

`LEDMATRIX_ANIMATION_HEARTBEAT`

`LEDMATRIX_ANIMATION_INFINITY_LOOP_LOADER`

`LEDMATRIX_ANIMATION_LOAD_CLOCK`

`LEDMATRIX_ANIMATION_LOAD`

`LEDMATRIX_ANIMATION_LOCK`

`LEDMATRIX_ANIMATION_NOTIFICATION`

`LEDMATRIX_ANIMATION_OPENSOURCE`

`LEDMATRIX_ANIMATION_SPINNING_COIN`

`LEDMATRIX_ANIMATION_TETRIS`

`LEDMATRIX_ANIMATION_WIFI_SEARCH`

Animation Generation

We have developed a tool that is used to generate frames and animations to be rendered on the LED Matrix in your browser. This tool is part of **Arduino labs**, and is therefore considered experimental software.

To use the tool you need to upload the following sketch, allowing the board to read serial

You can also find the sketch in **File > Examples > LED_Matrix > LivePreview**

```
#include "Arduino_LE

ArduinoLEDMatrix matr

void setup() {
  Serial.begin(115200
  matrix.begin();
}

uint32_t frame[] = {
  0, 0, 0, 0xFFFF
};

void loop() {
  if(Serial.available
    frame[0] = Serial
    frame[1] = Serial
    frame[2] = Serial
    matrix.loadFrame(
  }
}
```

Click here to go to the LED Matrix tool.

LED Matrix Editor

Once you've made your animations, you can export them from the tool in the format that lets you use them as **previously discussed**.

You can find more tips on how to use this tool on **its site**.

API

To write more advanced sketches on your own, you may use the full API of the library as found below.

Members	Descriptions
`public ArduinoLEDMatrix()`	The main class for controlling the LED matrix.
`public void autoscroll(int32_t interval_ms)`	Sets the time in ms for each frame to be displayed.
`public void begin()`	Start the LED matrix.
`public void next()`	Manually move to the next frame in the sequence.
`public void loadFrame(const uint32_t buffer[3])`	Load a new single frame that is not in any sequence.
`public void renderFrame(uint8:t frameNumber)`	Render the loaded frame.
`public void loadSequence(const uint32_t`	Loads an animation sequence into the buffer but does not display

Members	Descriptions
`public void play (bool loop = false)`	Start playing the sequence of frames, with the option to loop indefinitely or play once.
`public bool sequenceDone ()`	checks if the sequence has finished playing.

Members

`ArduinoLEDMatrix()`

Construct a new LED matrix object. This will be used to access the methods in the library.

```
ArduinoLEDMatrix LEDMa
```

`autoscroll()`

Enable autoscrolling through the frames in a sequence.

Parameters

`interval_ms` Sets the time in milliseconds that should be spent on a frame before switching to the next frame in the sequence.

Starts the LED matrix.

```
LEDMatrix.begin()
```

`next()`

Manually moves to the next frame in the sequence.

```
LEDMatrix.next()
```

`loadFrame()`

Loads a single frame that is not part of a sequence.

```
LEDMatrix.loadFrame(bu
```

Parameters

`buffer[3]` an array of three 32bit integers, where each bit represents an LED.

`renderFrame()`

Render a specific frame from a sequence.

```
LEDMatrix.renderFrame(
```

Parameters

`int` - frame to load.

`loadSequence()`

Loads an animation sequence into the buffer but does not display it.

```
LEDMatrix.frames[][4]
```

Parameters

`frameNumber` Specifies which frame of the sequence should be rendered.

`play()`

Starts playing the loaded sequence.

```
LEDMatrix.play(state)
```

Parameters

`loop` true to enable looping the sequence, false to play once.

`sequenceDone()`

Check for if the sequence is finished playing or if the frame should be advanced another

`false` if the sequence is not finished, `true` if it is.

Suggest changes

The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here.

Need support?

Help Center
Ask the Arduino Forum
Discover Arduino Discord

License

The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?